Latent Diffusion Models for Long-Horizon Autonomous Mobile Robot Planning

Shreepa Parthaje
Department of Engineering
University of Virginia, United States
shreepa@virginia.edu

Nicola Bezzo
Departments of Systems Engineering
United States
nbezzo@virginia.edu

Abstract: Recent works have framed autonomous mobile robot planning as a generative process via sampling trajectories from a diffusion model, producing high quality trajectories conditioned on a variety of multi-modal data. However, in long-horizon tasks, these methods are limited by planning in Euclidean space, as this requires significant compute resources. We introduce the *Latent Diffusion Planner* (LD-P) which instead generates trajectories entirely in a learned latent space. Our contributions are: i) a spatiotemporal autoencoder which compresses discretized trajectories by a factor of eight, and ii) a learned reverse diffusion process that generates trajectory latents conditioned on the physical coordinates of initial and goal states, resulting in both faster and fewer necessary denoising steps. Extensive simulation and experiment results are presented to validate the proposed LD-P as a planner for a quadrotor unmanned aerial vehicle (UAV) navigating through multiple gated course configurations, demonstrating generalization across environments. In these experiments, we generate 250 candidate plans, which are approximately 4s long as a part of a receding-horizon planning process. Furthermore, we show that LD-P is able to run at 43Hz on a laptop GPU, outperforming, by a factor of two, a baseline diffusion model sharing the same architecture but operating in Euclidean space. These results demonstrate that LD-P is an efficient, sampling-based planner capable of considering a large number of candidates in parallel for long-horizon mobile robot planning tasks.

Keywords: Latent Diffusion Models, Motion Planning, Unmanned Aerial Vehicle Applications

1 Introduction

Sampling-based motion planners are widely used in mobile robotics because they can simplify navigation of a complex multidimensional problem space to a stochastic sampling process. The challenge with sampling-based planners, however, is that the problem space has to be searched to find trajectories that both complete a goal and are optimal under some cost function. As a result, recent works consider diffusion models as a component in sampling-based planning because of their ability to model over the distribution of an expert collection of trajectories, reducing the planning process into two stages: sampling multiple goal-directed trajectories and choosing the lowest cost plan.

A significant body of work uses the reverse diffusion process to sample trajectories in Euclidean space, generating hf points in a trajectory, where h is the desired horizon in seconds and f is a control frequency [1, 2, 3]. While this approach allows for rich trajectory generation, it is limited primarily by inference speed and can struggle in mobile robotic applications where computational resources are a limiting factor. To address these limitations, more recent work has instead focused on generating an intermediate representation, such as control points for a B-spline [4], and then converting the intermediate into a trajectory at each denoising step for evaluation. These approaches, however, can require significantly more denoising steps to produce optimal intermediate behavior.

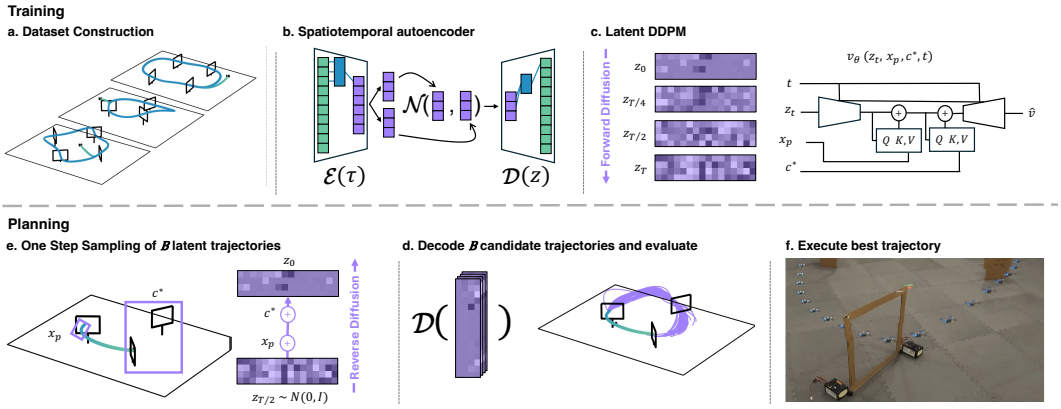


Figure 1: Overview of the proposed LD-P architecture. (a) Shows UAV navigation with trajectory data from three types of courses. (b-c) Illustrate the general architecture used to generate trajectory latents. During planning (e-f), the model samples trajectory latents conditioned on previous states and future waypoints, executing the highest scoring trajectory.

Inspired by latent diffusion models for image generation [5], we introduce the Latent Diffusion Planner (LD-P) which generates trajectories in a learned latent space. Rather than generating hf points, LD-P includes a spatiotemporal autoencoder which compresses these points, enabling faster inference while preserving the ability to evaluate the entire trajectory during the denoising process. We propose a conditioning mechanism that learns the residuals to the densest model state in the diffusion process, allowing both past states and goal waypoints to influence trajectory generation. This conditioning does not require temporal alignment between the input and conditioning signals, enabling 1) sampling of trajectories that reach a common waypoint at different times, and 2) trajectory generation that accounts for future waypoints even if they are not reached in the planning window. This information enables the planner to preemptively adjust a planned trajectory to better position the vehicle for passing future waypoints. Additionally, by representing all trajectories in a local frame, LD-P is able to make stronger generalizations in latent space with less data. Unlike Euclidean planners, LD-P can also generate trajectories in a single denoising step, reducing inference time significantly. This is critical in applications of high speed mobile robotics where planning has to be fast and on-device.

We consider one such application in quadrotor UAVs for navigating through laps in a series of gates. We demonstrate LD-P can generalize to unseen courses by leveraging local-frame symmetries and execute the planner at a high frequency. For this task, both the autoencoder and diffusion model are trained on trajectories from a few hundred courses across three course types (see Figure 1) for a total of approximately 25,000 trajectories collected in the gym environment from [6] for the Crazyflie platform. We then evaluate this planner in the real world on a Crazyflie with all computation done on a base station computer.

Our contributions in this work can be summarized as follows:

- A trajectory latent space that compresses Euclidean trajectories by a factor of eight and can reconstruct smooth position, velocity, and acceleration profiles
- A latent diffusion model that generates trajectory latents conditioned on data in Euclidean space using a residual conditioning mechanism that does not require temporal alignment
- Real-world validation on a UAV platform, showing sampling 250 candidate trajectories at 43 Hz

2 Related Works

Autoencoders for Trajectories Variational Autoencoders (VAEs) model data by encoding observations into a continuous latent space and decoding them to reconstruct the original input [7]. This framework has been adapted for encoding trajectories in road networks [8, 9, 10], where convolutional architectures process temporal or spatial patterns present. For example, [8] introduces a

smoothness term to penalize discontinuities in the generated trajectory in addition to the original reconstruction term. While these works focus on unconditioned trajectory generation, others explore planning in latent space. [11] encodes actions and observations jointly and samples policies conditioned on perceptual input. Similarly, [12] learns system dynamics in latent space for sampling-based planning, and [13] shows latent gradients can guide motion while avoiding obstacles. However, none of these approaches consider using diffusion models to sample latent trajectories which is the focus of our work.

Diffusion Planners and Policies Diffusion models are effective at generating data with a stochastic reverse diffusion process which iteratively removes noise added by a forward diffusion process [14, 15, 16, 17]. To accelerate this sampling process, [18, 19] propose a deterministic reverse process which requires fewer denoising steps. To improve sampling efficiency further, [5] proposed image generation in a learned latent space conditioned on a variety of multi-modal data during training. While these previous works primarily evaluated their approaches for image generation, diffusion models have also proven effective for both offline reinforcement learning and behavior cloning tasks. [20] introduced a temporal UNET architecture to model state-action trajectories across a horizon using a differentiable value function as a guide. [21] extends this to condition the model instead on an expected normalized reward, while [1] proposed using a transformer for a stronger ability to model high-frequency changes in actions opting to not model the states in the generated trajectory. More recent work builds on this to apply diffusion to robot planning [22, 23, 24, 25, 26, 27, 2, 28, 29, 30], including planning in a local frame [3, 31, 32] and using equivariance for better generalization from fewer samples [33, 34, 35]. However, these prior works have not explored latent diffusion models (LDMs) for planning and instead focus on modeling sequences as the full Euclidean trajectory, a series of waypoints, or more recently as a B-spline [4]. While LDMs have been applied to text-conditioned motion generation for humans [36, 37], our work is the first to study their use in mobile robot trajectory planning to leverage the faster sampling.

3 Background

3.1 Problem Statement

In this work, we study behavior cloning with a latent diffusion model for receding-horizon planning in mobile robot navigation. For navigation, we define a set of ordered waypoints $c = \{g_i\}_{i=0}^{G}$, where g_0 is a starting pose and each subsequent g_i is a waypoint in 3D space oriented with a heading θ. We define a trajectory as a discrete set of points in Euclidean space at a fixed control frequency: $\tau \in \mathbb{R}^{N \times 3}$. Then, the velocity and acceleration profiles can be defined as the first and second order derivatives of τ such that $\dot{\tau} \in \mathbb{R}^{N-1 \times 3}$ and $\ddot{\tau} \in \mathbb{R}^{N-2 \times 3}$. To refer to a specific 3D point in the horizon, we denote it as $\tau(j)$, and to refer to a specific dimension in the horizon we denote it as $\tau(j, s)$, where $s \in \{0, 1, 2\}$.

Since this navigation task concerns pairs of waypoints $(g_i, g_j) \in SE(3) \times SE(3)$, the valid set of trajectories is invariant to simultaneous $SE(3)$ transformations applied to both the trajectories and the waypoints. This invariance can be leveraged for stronger generalization by planning in a local frame, centered and aligned with the start of the trajectory. We use diffusion models within a sampling-based planning framework due to their ability to model the distribution over an expert set of trajectories. We specifically look to latent diffusion models because it significantly reduces computation time by compressing the horizon of a trajectory and by requiring fewer denoising steps during sampling. This requires learning both a generator $p(\tau|z)$ and a sampler $q(\mu_z|c^*, x_p)$, where x_p is the p most recent states of the robot, c^* is the next set of waypoints, and μ_z is a point estimate of $q(z|\tau)$. We learn an encoder-decoder pair $(\mathcal{E}, \mathcal{D})$, such that $\mathcal{D}(\mathcal{E}(\tau)) \approx \tau$, to construct a latent space where $\mathcal{D}$ can generate trajectories from a point estimate of the latent. We then train a diffusion model to approximate sampling from the posterior $q(\mu_z|c^*, x_p)$. In the notation moving forward, we will denote the sampling process of the point estimate as $q(z|c^*, x_p)$. For this sampler to be effective, it must produce dynamically feasible trajectories which are continuous with previous states and be goal-directed (i.e. either passing waypoints or positioning the robot to pass waypoints in future

planning cycles). With a diverse set of valid trajectories from the sampler, we can define multiple simple scoring functions to favor specific behavior at runtime (e.g. speed, curvature, etc.). This modular design allows flexible integration of new objectives at runtime built on the same latent sampling foundation.

3.2 Latent Diffusion Models (LDMs) and Variational Autoencoders (VAEs)

In our work, we choose to progressively add Gaussian noise ϵ to a trajectory latent z via a forward diffusion process over T timesteps, such that z_0 is the original latent and z_T approaches pure noise [14, 15, 5]. The reverse diffusion's objective is to iteratively predict the noise ϵ given z_t and t, remove this ϵ, and add back in a newly sampled ϵ at decreasing amounts for a fully stochastic sampling procedure across T steps. Treating the reverse objective as a deterministic solver, however, means steps can be skipped for $S < T$ total denoising iterations, accelerating the inference process [18, 19]. In our work, we leverage this deterministic sampling over fewer steps to sample trajectory latents and plan at a higher frequency.

We use variational autoencoders (VAEs) to encode τ into a continuous latent variable z, since the VAE decoder is trained to reconstruct τ from samples of $q(z|\tau)$ rather than fixed point estimates [7]. When the VAE prior is chosen to be Gaussian, this is achieved by training an encoder $\mathcal{E}$ to estimate the mean and log-variance of $q(z|\tau)$ and training the decoder $\mathcal{D}$ to reconstruct τ from $\mu_z + \sigma_z\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$. To prevent $q(z|\tau)$ from being pushed towards a Dirac delta distribution, a regularization term is also added via a Kullback–Leibler divergence which measures the distance between the posterior and the prior $p(z)$. The stochastic training objective plus regularization term makes the decoder more robust to residual noise introduced by using fewer denoising steps in the reverse diffusion process.

4 Method

Our approach can be broken into the design of the trajectory latent space and the conditioning mechanism for the latent diffusion model. As identified in Section 3.1, all trajectories are expected to be in a local frame centered at and aligned with the start of the trajectory defined as an SE(3) transformation. We choose to model both the autoencoder and diffusion priors as Gaussian distributions because of tractability and a significant body of work showing success with this choice [5, 9, 20, 1].

4.1 Trajectory VAE

We modify the temporal UNET architecture from [20] by removing the skip connections to form our encoder / decoder blocks to operate over the time horizon H of a given τ. At the final layer of the encoder, the model state $\in \mathbb{R}^{H/8\times 1024}$ is projected onto $\mathbb{R}^{H/8\times 8}$ and split into two $\mathbb{R}^{H/8\times 4}$ tensors to represent $(\mu_z, \log\sigma_z^2)$ the the parameters of $q(z|\tau)$. The decoder is trained to reconstruct τ from a random sample from $q(z|\tau)$ by the following reparameterization: $\mu_z + \sigma_z\epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$. Since these trajectory latents are used in a receding-horizon planner, encoding the initial states of the trajectory is crucial for continuity and decoding a smooth velocity and acceleration profile is a necessity for tracking. To this end, we replace the reconstruction term of the loss with Equation 1, where d is a distance function (e.g. L1 loss in our experiments), $\hat{\tau}_i = \mathcal{D}(\mathcal{E}(\tau_i)) = \mathcal{D}(\mu_{\tau_i} + \sigma_{\tau_i}\epsilon)$, and w is a function that defines weighting of the loss term along the horizon :

$$\mathcal{L}_{recon} = \frac{1}{3H}\sum_{j=1}^{H}\sum_{s=1}^{3} w(j)\left(d\left(\tau_i(j,s) - \hat{\tau}_i(j,s)\right) + \sum_{k=1}^{K}\zeta_k\, d\left(\frac{d^k}{dt^k}\tau_i(j,s) - \frac{d^k}{dt^k}\hat{\tau}_i(j,s)\right)\right) \quad (1)$$

In our training we smooth up to $K = 2$ order and weight the beginning of the trajectory higher in the loss computation with $w(j)$. We found this allows the trajectory latent to not only reconstruct the initial dynamics more accurately, but also allows a diffusion sampler to better adjust for continuity with prior states x_p. Also since the Δt can be factored out of derivative of τ_i and $\hat{\tau}_i$, we abstract it away as a part of ζ_k and only compute repeated pairwise differences for the derivatives. We find smoothing past the 2nd order offers minimal benefits and creates training instability. The motivation

for this loss extends the idea of first order smoothness factor in [8] and the idea of weighting noise estimates in a diffusion model in [20]. The total loss formulation for training the autoencoder also includes a KL-Divergence term for regularization weighted by β: $\mathcal{L} = \mathcal{L}_{recon} + \beta\mathcal{L}_{KL}$. With this loss formulation, the position and velocity profiles are smooth, but the slight errors in the reconstructed trajectory manifest as significant noise in the acceleration profile created by differentiating $\hat{\tau}$. To reconstruct the acceleration profile we apply a Savitzky-Golay filter [38] on the raw acceleration profile which we show is centered around the true acceleration. See Appendices A.1 and A.2 for details on the effect of this loss formulation and the filter.

4.2 Latent Diffusion Model and Residual Conditioning

To generate continuous, goal-directed trajectories, we sample trajectory latents from a Latent Diffusion Model (LDM). Our LDM architecture builds on the temporal UNET architecture introduced in [20], adapting it to operate over the compressed latent horizon. Crucially, we incorporate a residual conditioning mechanism inspired by Stable Diffusion [5] to flexibly guide generation without hard temporal alignment (i.e. conditioning on specific points in the trajectory).

Conditioning Mechanism: We use the conditioning mechanism to produce trajectories which are continuous with the current state of the robot and aligned with future waypoints. To enable this, we condition the diffusion model on two sources of information: $x_p \in \mathbb{R}^{p\times 3}$ which is the robot's p prior positions and $c^* \in \mathbb{R}^{N\times 4}$ which is up to the next N waypoints with both position and heading. We use a multilayer perceptron (MLP) to project this information into the model's internal embedding space. To retain the temporal structure of the prior states and the order of future waypoints, we add sinusoidal positional encodings [39] before injecting these embeddings into the model. While Stable Diffusion applies this conditioning at every layer of the UNET, we find applying it at the bottleneck of the model where the temporal resolution is lowest is sufficient. We use two cross-attention layers at the bottleneck for each source of conditioning and use the attention output as a residual update to the bottleneck features. This residual learning enables the model to effectively condition on the prior states and future goals while at lowest temporal resolution where each token represents approximately $1s$ along the horizon, significantly reducing computational overhead. This method of conditioning avoids the rigid temporal alignment of inpainting-style approaches [20, 23, 31, 3, 27, 2], allowing for the planner to effectively choose the time to travel between waypoints. It also means future waypoints which aren't passed can still influence the planning process to better position the robot for the next iteration of planning.

Training Objective We train this LDM with the objective of predicting v which is a linear combination of ϵ and z_0, as first proposed by [40] where $v_t = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1-\bar{\alpha}_t}z_0$. Training on this objective results in better training stability and empirically higher consistency when planning. The intuition behind this is at the final diffusion timestep the signal to noise ratio (SNR) of z_T is very low, whereas v_T maintains a high SNR. Conversely, at the early timesteps where the SNR of z_t is high, v_t maintains a low SNR, capturing any remaining noise. As propsed by [41], this means we do not have to clip the SNR to be greater than zero in training as originally required for ϵ-prediction in [42]. We further explore the benefit of this training objective in Appendix A.3.

Sampling and Classifier-Free Guidance: Once the LDM is trained, we can sample trajectory latents from it using a DDIM solver [18] on the v-predictions [40, 41]. While sampling, we can also opt to use classifier-free guidance [43] to strengthen the signal from x_p and c^* by defining $v^*_\theta = (1+w)v_\theta(z_t, x_p, c^*, t) - wv_\theta(z_t, x_p, \phi_{c*}, t)$ where ϕ_{c*} is the null token which is equivalent to not conditioning on any future waypoint. When sampling trajectory latents, we define a one-step approximation as $z_0 \approx \sqrt{\bar{\alpha}_{T/2}} \cdot z_{T/2} - \sqrt{1-\bar{\alpha}_{T/2}} \cdot v^*_\theta$ which can generate high quality trajectories that the Euclidean diffusion model requires multiple DDIM steps for We do this by approximation that the SNR at $t = T/2$ is zero, meaning $z_{T/2} \sim \mathcal{N}(0, I)$.

4.3 Planning with LD-P

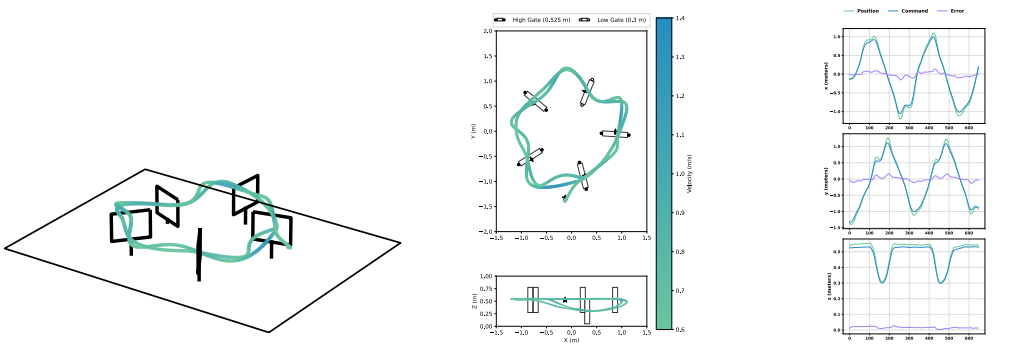
LD-P plans in a local frame of reference centered and aligned with the first point of the trajectory building off of [3, 31]. Since the initial position along the plan is unknown, we can use forward kinematics on the previous state to approximate this first point: $\hat{x}_0 = x_{-1} + \dot{x}_{-1}\Delta t + \frac{1}{2}\ddot{x}_{-1}\Delta t^2$. We can then use the vector between x_{-1} and $\hat{x}_0$ to approximate the heading of the trajectory. To increase the amount of symmetries in the dataset, when the vehicle is not moving (so $x_{-1} \approx \hat{x}_0$), we find aligning the local frame with the next waypoint improves performance. Critically, when converting to this local frame the rotation of the system has to be applied to the orientation of the waypoints as well. With these transformations, the conditioning for the LDM can also be transformed to get c^* and x_p relative to the next generated trajectory. The effect of exploiting these symmetries on the conditioning in local frame is further explored in Appendices A.4 and B. With these transformations, we can use the sampling step from the previous section to generate a large quantity of trajectory latents. These are then decoded, smoothed, and filtered to ensure they pass through the immediate waypoint. A simple scoring heuristic is then sufficient to favor certain behavior and pick the best trajectory in the set. In our experiments, we cut the trajectory at the next waypoint and then re-plan from this state although the model is trained for any x_p conditioning.

4.4 Application of LD-P to UAV Navigation

To validate LD-P on a mobile robot platform, we consider UAV navigation through gates for the Crazyflie platform as seen in Figure 1. To construct our dataset we define three types of courses (rhombus, triangle, and oval seen left to right in Figure 1a) and generate 322 variations of these courses with a 14%, 68%, and 38% split between the three types. The courses vary in the angle between each gate, the angle of the gate itself, and the distance of the gate from the origin, resulting in both clockwise and counter-clockwise gate setups. For each course, we aim to sample 50-100 random fifth-order spline trajectories which pass through each gate, completing a lap and validate these trajectories within simulation [6] using a Mellinger Controller [44]. We also add an empirically evaluated constraint on the acceleration term to address a gap between what is feasible in simulation and real life: $\|\ddot{\tau}(t)\|_2 < -0.3(\|\dot{\tau}(t)\|_2)^3 + 2.0$. When constructing our dataset, we only consider SE(2) symmetries and leave the z-position in the shared world frame because there are no symmetries along the z-axis to take advantage of for this task. We further elaborate this in Appendix C. For this planning task, the decoder applies a Savitzky-Golay filter with a third-order polynomial over a 0.66-second window to the acceleration profile, as quadrotor UAVs are capable of tracking cubic (third-order) acceleration trajectories effectively.

5 Results

5.1 LD-P in Unseen Environments



(a) Perspective of Navigation in Pentagon (b) UAV States in Simulation (c) XYZ State Errors

Figure 2: Simulation of LD-P on an unseen course type with one diffusion step.

To evaluate how this approach generalizes across all environments, we construct a set of 85 evaluation courses and run the planner to complete two laps through these courses, sampling $B = 250$

trajectories at each gate. We use the same process from Section 4.4 to construct 25 new courses per type seen in training (rhombus, triangle, and oval). We also create two new types of courses which feature five and six gates arranged in a pentagon and hexagon pattern respectively for an additional 10 courses. We evaluate our LD-P approach against a baseline diffusion planner (D-P) which shares our architectural contributions but operates in Euclidean space. To asses the continuity of the planner with the current trajectory, we define a metric which measures the difference in the expected initial velocity and the actual initial velocity: $\Delta EV = \|\hat{\dot{x}}_0 - \dot{x}_0\|_2$. We also measure the percentage of trajectories passing the filtering stage (candidate percentage) and variety in the set to measure the diversity of sampled trajectories. The variety metric is defined as the average pairwise difference between any two valid trajectories in the batch at each timestep $\text{Variety} = \frac{1}{H}\sum_{j=1}^{H}\left(\frac{1}{\binom{B}{2}}\sum_{1\leq i<k\leq B}\|\tau^i(j) - \tau^k(j)\|_2\right)$ where H is the number of points in the horizon and B is the number of samples considered.

Table 1: Sample quality across 85 evaluation courses and different number of denoising steps comparing LD-P to a baseline diffusion planner

	Steps	ΔEV $(ms^{-2})\downarrow$	Candidates (%) $\uparrow$	Variety $(m)\uparrow$	Success (%)	Max Hz
D-P	10	0.04 ± 0.006	36 ± 0.4	0.13 ± 0.06	100.0	5.7 ± 0.01
LD-P	10	0.06 ± 0.02	36 ± 5.0	0.13 ± 0.05	92.94	15.04 ± 0.5
D-P	5	0.06 ± 0.007	36 ± 0.07	0.12 ± 0.05	**100.0**	$\mathbf{11.1 \pm 0.05}$
LD-P	5	0.06 ± 0.02	36 ± 5.0	0.12 ± 0.05	94.12	24.9 ± 1.12
D-P	3	0.27 ± 0.06	34 ± 3.1	0.12 ± 0.03	90.58	15.16 ± 0.45
LD-P	3	0.07 ± 0.02	35 ± 7	0.13 ± 0.05	90.58	32.86 ± 2.04
D-P	2	-	-	-	0.0	-
LD-P	2	0.07 ± 0.02	32 ± 8	0.14 ± 0.06	91.76	38.88 ± 2.98
D-P	1	-	-	-	0.0	-
LD-P	1	0.07 ± 0.03	36 ± 7	0.14 ± 0.06	**91.76**	$\mathbf{46.73 \pm 3.76}$

Our results from Table 1 show that LD-P can sample trajectories in just 1 step achieving >90% success rate on the evaluation set where the baseline needs three diffusion steps to achieve similar success at a significantly lower sample quality (ΔEV). Notably, we show our light-weight conditioning approach is enough to reach a 100% success rate in five denoising steps in Euclidean space, and that our proposed LD-P can run at 47 Hz with a relatively high success rate. We also highlight that planning in latent space minimally impacted sample quality ΔEV and did not impact sample diversity. Of the seven courses the one-step LD-P failed on, three were hexagon courses and four rhombus courses. We hypothesize that this occurred because these courses required motions which are significantly underrepresented in the dataset (i.e. swings in z over short lateral distances and high curvature movement). Figure 2 shows the one step LD-P successfully navigating a pentagon course from this evaluation set for two laps in simulation with minimal tracking error. We find across these experiments, the controller [44] tracks with approximately $0.05m$ of Euclidean error in simulation.

5.2 Scoring Heuristic Effect on Planning

With a diverse set of trajectories generated by the one-step latent sampling procedure, we expect there to be specific behaviors we can favor via a scoring objective. To validate this hypothesis in simulation, we define four heuristics for navigation and identify their effect with a proxy measurement averaged over eight laps in a fixed oval course. From the sample of 4 second trajectories, these heuristics choose the trajectory that either covers the most distance (minimizing lap time), covers the least distance (maximizing lap time), maximizes safety (minimizes distance from a gate's center when passing), or has the lowest maximum curvature.

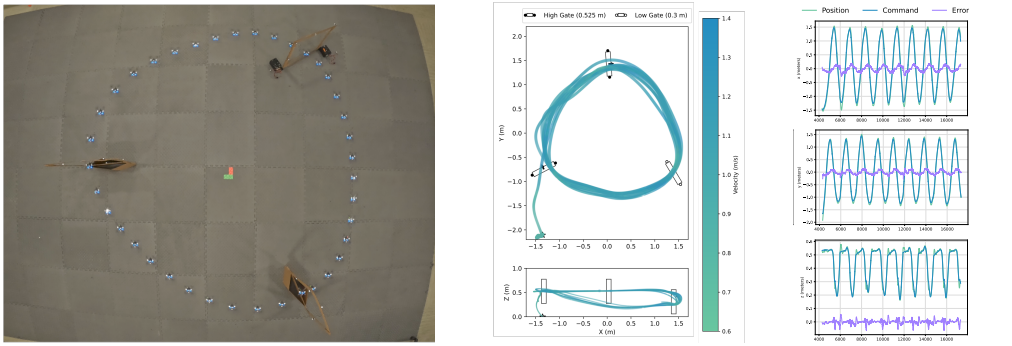
Table 2: Evaluating effect of different scoring heuristics on a 8 lap navigation of an oval course

	Lap Time (s)	Distance from Gate Center (m)	Max-Curvature (m^{-1})
Longest Trajectory	$\mathbf{6.525 \pm 0.420}$	0.08 ± 0.040	10.13 ± 45.13
Shortest Trajectory	$\mathbf{8.77 \pm 0.457}$	0.08 ± 0.034	11.60 ± 28.20
Safest Trajectory	8.16 ± 0.140	$\mathbf{0.031 \pm 0.004}$	8.70 ± 29.02
Low Curvature Trajectory	7.42 ± 0.732	0.075 ± 0.036	$\mathbf{2.86 \pm 2.19}$

Table 2 successfully illustrates the diversity of samples generated by observing the difference in the metrics each heuristic optimizes. For lap time, the competing heuristics can create over a 2-second difference in average lap time. When choosing the safest trajectory, we see that there is up to $0.05m$ added margin from a gate's edge. Similarly, when choosing to minimize curvature, the maximum curvature along the trajectories are up to three times lower than the highest. These results highlight the effectiveness of simple heuristics combined with sampling a large quantity of diverse trajectories.

5.3 UAV Experiments

For real world experiments, we localize the UAV and the gates using a Vicon system. We build the online planning algorithm using ROS and the crazyswarm package [45] to communicate with a Crazyflie 2.0 UAV and send states for the controller [44] to track. As seen in Figure 3, we analyze an experiment where the UAV completes 8 laps around a triangular course. A one-step LD-P, using a minimizing lap-time heuristic, completed laps in $7.43s \pm 0.15s$ with an average speed of $1.11ms^{-1} \pm 0.24ms^{-1}$, requiring a similar computation time as reported in simulations in Table 1.



(a) Sequence of snapshots for one completed lap

(b) The overall path from different perspectives

(c) Vicon observations vs command

Figure 3: Experiment results on a Crazyflie quadrotor in an indoor 3 gates course.

6 Discussion

In this work, we study the use of Latent Diffusion Models for receding-horizon planning in mobile robotics as an alternative to sampling in Euclidean space. We propose LD-P to generate smooth position, velocity, and acceleration profiles that are continuous with a robot's current state and navigate through desired waypoints via the culmination of a spatiotemporal autoencoder, latent diffusion model, and scoring heuristic. Through a light conditioning mechanism, we show that LD-P can leverage information about waypoints without hard temporal alignment and also use future waypoints to better position robots at the end of the planning horizon. We also show when using this conditioning mechanism, we can sample a large quantity of candidate trajectories and then use simple scoring heuristic to maximize an axillary objective. In both simulation and on a real system, we validate the proposed planner on a UAV platform which navigates through gates and completes laps, showing it can match the performance of a baseline Euclidean diffusion model while sampling significantly faster.

7 Limitations

A limitation of this work is the requirement on a diverse dataset to learn from compared to a baseline diffusion model. In the UAV navigation task, while the latent diffusion model could generalize to new navigation tasks, it struggled when required to generalize new motion primitives as well. The hexagon courses, for instance, require the UAV to bob up and down through three or four gates in the same amount of lateral distance that could be covered by two gates in a triangular course. We hypothesize not having seen high z velocities for relatively low x and y velocities in latent space caused the planner to struggle. Another limitation is this work does not address collision-aware planning in latent space and only focuses on navigation in latent space. Incorporating collision-aware planning in latent space has been explored by [13], and doing so with a diffusion planner is a promising future direction. The last limitation is from the requirement of an "expert" dataset to see expert behavior cloned by the diffusion model. This could be addressed by using reinforcement learning for diffusion models [46] to reward expert behavior.

Acknowledgments

If a paper is accepted, the final camera-ready version will (and probably should) include acknowledgments. All acknowledgments go at the end of the paper, including thanks to reviewers who gave useful comments, to colleagues who contributed to the ideas, and to funding agencies and corporate sponsors that provided financial support.

References

[1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.

[2] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters. Motion planning diffusion: Learning and planning of robot motions with diffusion models. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1916–1923. IEEE, 2023.

[3] W. Yu, J. Peng, H. Yang, J. Zhang, Y. Duan, J. Ji, and Y. Zhang. Ldp: A local diffusion planner for efficient robot navigation and collision avoidance. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5466–5472. IEEE, 2024.

[4] J. Carvalho, A. Le, P. Kicki, D. Koert, and J. Peters. Motion planning diffusion: Learning and adapting robot motion planning with diffusion models. *arXiv preprint arXiv:2412.19948*, 2024.

[5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[6] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig. Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics. *IEEE Robotics and Automation Letters*, 7(4):11142–11149, 2022.

[7] D. P. Kingma, M. Welling, et al. Auto-encoding variational bayes, 2013.

[8] O. Rákos, S. Aradi, T. Bécsi, and Z. Szalay. Compression of vehicle trajectories with a variational autoencoder. *Applied Sciences*, 10(19):6739, 2020.

[9] O. Rákos, T. Bécsi, S. Aradi, and P. Gáspár. Learning latent representation of freeway traffic situations from occupancy grid pictures using variational autoencoder. *Energies*, 14(17):5232, 2021.

[10] X. Chen, J. Xu, R. Zhou, W. Chen, J. Fang, and C. Liu. Trajvae: A variational autoencoder model for trajectory generation. *Neurocomputing*, 428:332–339, 2021.

[11] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet. Learning latent plans from play. In *Conference on robot learning*, pages 1113–1132. Pmlr, 2020.

[12] B. Ichter and M. Pavone. Robot motion planning in learned latent spaces. *IEEE Robotics and Automation Letters*, 4(3):2407–2414, 2019.

[13] J. Yamada, C.-M. Hung, J. Collins, I. Havoutis, and I. Posner. Leveraging scene embeddings for gradient-based motion planning in latent space. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5674–5680. IEEE, 2023.

[14] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.

[15] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[16] Y. Song and S. Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

[17] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[18] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[19] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.

[20] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.

[21] A. Ajay, Y. Du, A. Gupta, J. Tenenbaum, T. Jaakkola, and P. Agrawal. Is conditional generative modeling all you need for decision-making? *arXiv preprint arXiv:2211.15657*, 2022.

[22] U. A. Mishra, S. Xue, Y. Chen, and D. Xu. Generative skill chaining: Long-horizon skill planning with diffusion models. In *Conference on Robot Learning*, pages 2905–2925. PMLR, 2023.

[23] Z. Wang, T. Oba, T. Yoneda, R. Shen, M. Walter, and B. C. Stadie. Cold diffusion on the replay buffer: Learning to plan from known good states. In *Conference on Robot Learning*, pages 3277–3291. PMLR, 2023.

[24] Y. Wang, Y. Zhang, M. Huo, R. Tian, X. Zhang, Y. Xie, C. Xu, P. Ji, W. Zhan, M. Ding, et al. Sparse diffusion policy: A sparse, reusable, and flexible policy for robot learning. *arXiv preprint arXiv:2407.01531*, 2024.

[25] Z. Liang, Y. Mu, H. Ma, M. Tomizuka, M. Ding, and P. Luo. Skilldiffuser: Interpretable hierarchical planning via skill abstractions in diffusion-based task execution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16467–16476, 2024.

[26] X. Liu, Y. Zhou, F. Weigend, S. Sonawani, S. Ikemoto, and H. B. Amor. Diff-control: A stateful diffusion-based policy for imitation learning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7453–7460. IEEE, 2024.

[27] S.-W. Guo, T.-C. Hsiao, Y.-L. Liu, and C.-Y. Lee. Precise pick-and-place using score-based diffusion networks. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7484–7491. IEEE, 2024.

[28] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki. 3d diffuser actor: Policy diffusion with 3d scene representations. *arXiv preprint arXiv:2402.10885*, 2024.

[29] H. Huang, B. Sundaralingam, A. Mousavian, A. Murali, K. Goldberg, and D. Fox. Diffusionseeder: Seeding motion optimization with diffusion for rapid motion planning. *arXiv preprint arXiv:2410.16727*, 2024.

[30] N. Botteghi, F. Califano, M. Poel, and C. Brune. Trajectory generation, control, and safety with denoising diffusion probabilistic models. *arXiv preprint arXiv:2306.15512*, 2023.

[31] J. Liang, A. Payandeh, D. Song, X. Xiao, and D. Manocha. Dtg: Diffusion-based trajectory generation for mapless global navigation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5340–5347. IEEE, 2024.

[32] B. Yang, H. Su, N. Gkanatsios, T.-W. Ke, A. Jain, J. Schneider, and K. Fragkiadaki. Diffusion-es: Gradient-free planning with diffusion for autonomous and instruction-guided driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15342–15353, 2024.

[33] D. Wang, S. Hart, D. Surovik, T. Kelestemur, H. Huang, H. Zhao, M. Yeatman, J. Wang, R. Walters, and R. Platt. Equivariant diffusion policy. *arXiv preprint arXiv:2407.01812*, 2024.

[34] J. Brehmer, J. Bose, P. De Haan, and T. S. Cohen. Edgi: Equivariant diffusion for planning with embodied agents. *Advances in Neural Information Processing Systems*, 36:63818–63834, 2023.

[35] J. Yang, Z.-a. Cao, C. Deng, R. Antonova, S. Song, and J. Bohg. Equibot: Sim (3)-equivariant diffusion policy for generalizable and data efficient learning. *arXiv preprint arXiv:2407.01479*, 2024.

[36] X. Chen, B. Jiang, W. Liu, Z. Huang, B. Fu, T. Chen, and G. Yu. Executing your commands via motion diffusion in latent space. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18000–18010, 2023.

[37] W. Dai, L.-H. Chen, J. Wang, J. Liu, B. Dai, and Y. Tang. Motionlcm: Real-time controllable motion generation via latent consistency model. In *European Conference on Computer Vision*, pages 390–408. Springer, 2024.

[38] A. Savitzky and M. J. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical chemistry*, 36(8):1627–1639, 1964.

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[40] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.

[41] S. Lin, B. Liu, J. Li, and X. Yang. Common diffusion noise schedules and sample steps are flawed. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 5404–5411, 2024.

[42] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pages 8162–8171. PMLR, 2021.

[43] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[44] D. Mellinger and V. Kumar. Minimum snap trajectory generation and control for quadrotors. In *2011 IEEE international conference on robotics and automation*, pages 2520–2525. IEEE, 2011.

[45] J. A. Preiss*, W. Hönig*, G. S. Sukhatme, and N. Ayanian. Crazyswarm: A large nano-quadcopter swarm. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3299–3304. IEEE, 2017. doi:10.1109/ICRA.2017.7989376. URL https://doi.org/10.1109/ICRA.2017.7989376. Software available at https://github.com/USC-ACTLab/crazyswarm.

[46] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.

A Ablation Study

A.1 Effect of Reconstruction Loss on Autoencoder

We define the reconstruction loss in Section 4.1 to reconstruct a smoother velocity and acceleration profile. To understand the effects of the smoothing and weighted term, we train a separate model on an L1 reconstruction objective. From Figures 4 and 5, we can empirically observe two effects of this loss function. In both residual plots, the magnitude of the errors are much larger (with the acceleration errors reaching up to $6ms^{-2}$ of error) in the L1 objective whereas when trained with the smoothness objective we see errors of much smaller magnitude. Furthermore, we see that although the error in velocity and acceleration profile spike relatively near the last timestep in both models, only the model trained on the L1 objective sees the same increase in error at the early points of the trajectory. We believe the weighting along the horizon helps better represent early dynamics which is critical for planning from an initial state.

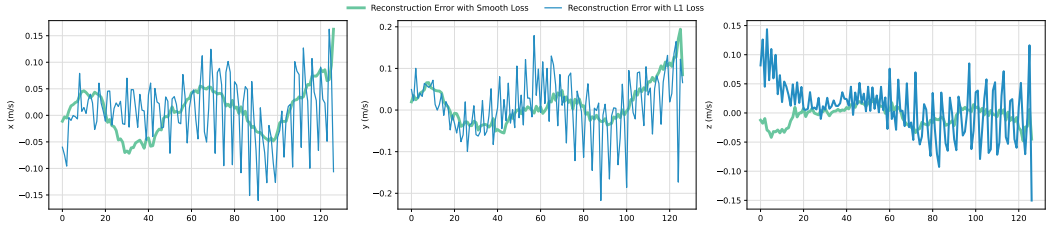


Figure 4: Given a random sample τ, we reconstruct its velocity profile $\dot{\tau}$ using a encoder-decoder pair trained on the loss formulation presented in this paper (green) versus a pure distance based reconstruction loss (L1 in this case) used in standard autoencoders (blue).

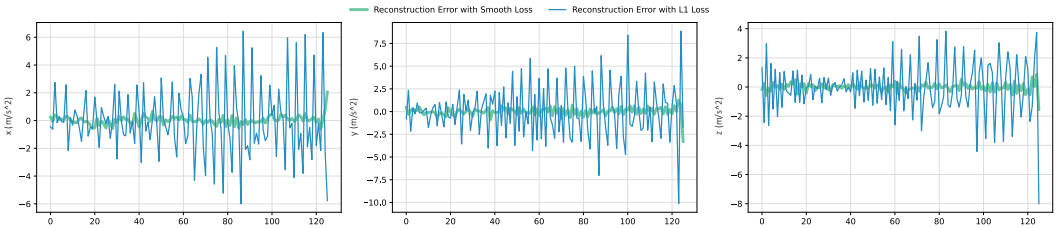


Figure 5: Given a random sample τ, we reconstruct its acceleration profile $\ddot{\tau}$ using a encoder-decoder pair trained on the loss formulation presented in this paper (green) versus a pure distance based reconstruction loss (L1 in this case) used in standard autoencoders (blue).

A.2 Effect of Smoothing Function on Acceleration Reconstruction

In this section we observe what happens to the acceleration error term with and without the Savitzky–Golay filter applied in Figure 6 . Although the filter does not produce a perfectly smooth reconstruction of the acceleration profile, we see that it helps capture the underlying data.

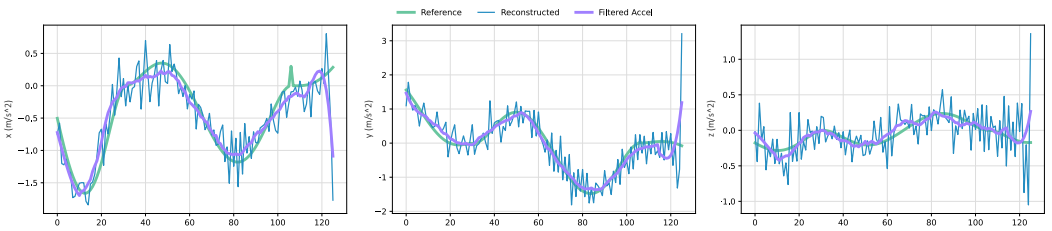


Figure 6: Random sample $\ddot{\tau}$ from the autoencoder dataset shown in green versus the reconstructed $\ddot{\hat{\tau}}$ shown in blue. Shown in purple is the reconstruction after applying a filter to remove reconstruction noise.

A.3 Choosing a Diffusion Objective: ϵ vs. v

Diffusion models are primarily trained on ϵ-prediction from [15] which estimates the amount of noise added to a sample. As discussed in Section 4, we instead choose to train on v-prediction [40] and also do not clip the SNR [41]. To observe the effect of these decisions, we also train latent diffusion models to predict ϵ, v where the SNR is clipped, and z_0 directly. We evaluate the success rate of each of these models with different number of denoising steps to compare against the results from Section 5 . From Table 3, we that our choices in training objective lead to higher success rates across the different number of denoising steps compared to the alternative objectives.

Table 3: Success rate across 85 evaluation courses and different number of denoising steps, comparing different diffusion objectives

	10	5	3	2	1
v	**92.94**	**94.12**	**90.58**	**91.76**	**91.76**
v SNR-clipped	85.88	87.05	87.05	90.58	85.88
ϵ	89.41	88.23	89.41	87.05	85.88
z_0	89.41	89.41	87.05	84.70	81.17

A.4 Planning in Trajectory Frame vs. Shared World Frame

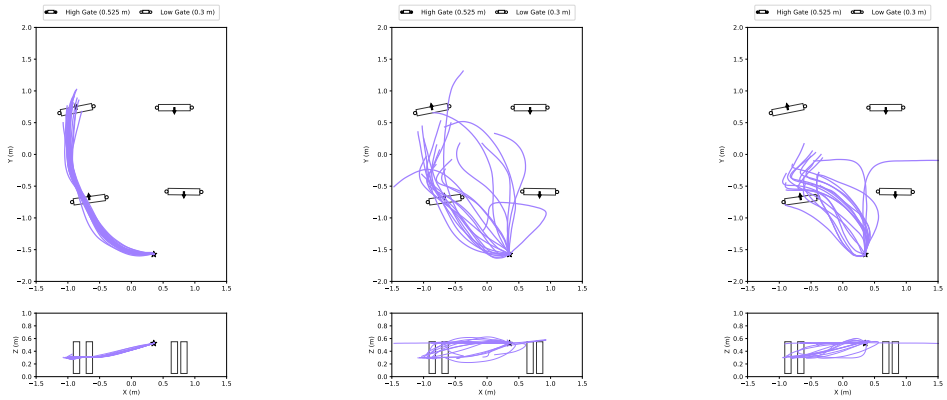
We train the proposed LD-P on the UAV navigation task with both data in local frame and the same data in world frame to analyze if leveraging symmetries in the local frame improves model generalization. From Table 4, we see the LD-P World cannot break 80% success rate on our evaluation set from 5. From the 10 new courses introduced, we see at maximum only 20% get traversed successfully for two laps, suggesting the world frame requires significantly more data.

Table 4: Success rate across 85 evaluation courses and different number of denoising steps, comparing LD-P to LD-P in world frame

	10	5	3	2	1
LD-P	**92.94**	**94.12**	**90.58**	**91.76**	**91.76**
LD-P World	71.76	70.59	75.29	74.11	74.11

A.5 Residual Conditioning

In this section, we examine the residual conditioning mechanism in the UAV navigation task by examining the effect of it when it is removed completely (i.e. that signal pathway is turned off) and when c^* is replaced with the null token ϕ_{c*}. From Figure 7, we see that in the presence of c^* the trajectories not only pass through the first gate, but also the second and prepare to arc around to get to the third. We see that the generated trajectories share strong resemblances in the presence of the null token and when the global conditioning pathway is off. This is an effect of residual learning, where the model approximates the residual from ϕ_{c*} is zero which is essentially the same as the pathway being disabled.

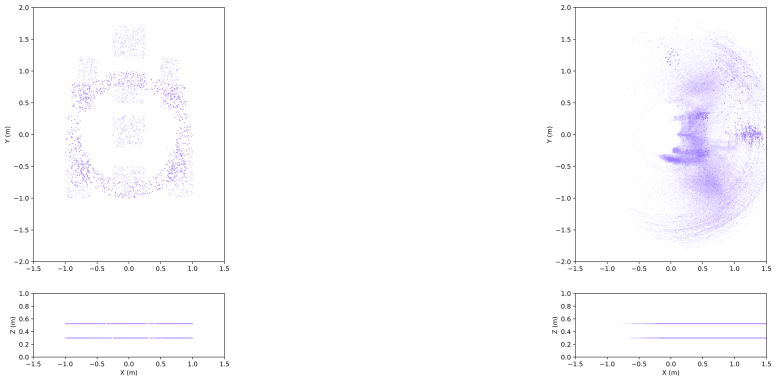


(a) Conditioning on x_p and c^* (b) Conditioning on x_p and ϕ_{c*} (c) Conditioning only on x_p

Figure 7: Generating 20 trajectories conditioned on varying inputs

B Exploring Symmetries in UAV Dataset

To get a visual understanding what symmetries can be exploited by planning in a local frame, we plot the first waypoint in c^* from our UAV navigation dataset in both world frame and local frame in Figure 8. From this, it is visible how most of these waypoints fall along the same arc in the local frame suggesting similar planning tasks which would look vastly different in the world frame under rotation.



(a) First waypoint in dataset in world frame (b) First waypoint in dataset in local frame

Figure 8: First waypoint from all c^* in the UAV dataset in both the world and local frame

C Dataset Samples for UAV Navigation

In Section 4, we describe how we consider only SE(2) symmetries because it helped the reconstruction of the trajectory for the UAV task specifically. Figure 9 illustrates why this is the case by showing a set of trajectories in the defined trajectory frame. Along the z-axis, there is no meaningful symmetry to exploit with only four distinct modes. If we assume these modes are equally distributed, then when planning in the trajectory frame, they effectively collapse into just three modes, with one mode (maintaining the same altitude) accounting for half of the data. As a result, applying an SE(3) transformation degrades reconstruction quality in this context, which explains the poor performance

when using SE(3) instead of SE(2). We can see what the diffusion input data looks like in the UAV trajectory frame in Figure 10.

C.1 Autoencoder

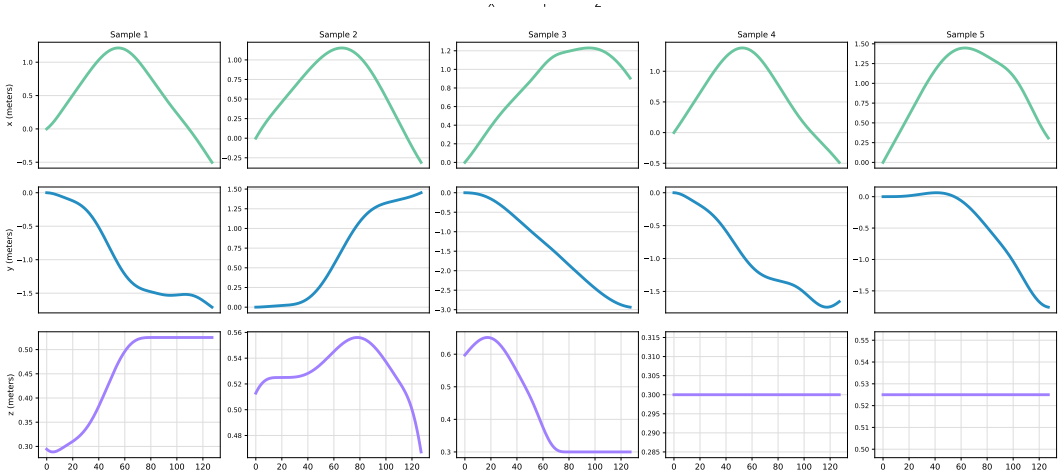


Figure 9: Five examples from the dataset used to train the spatiotemporal autoencoder in the shared trajectory frame. Note the four modes in the z-axis (low-to-high, high-to-low, low, high) pictured in these samples.

C.2 Latent Diffusion Model

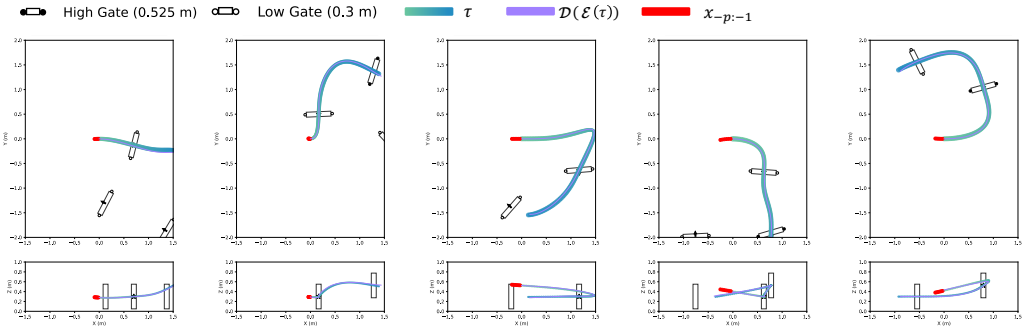


Figure 10: Five examples from the dataset used to train the latent diffusion model in the shared trajectory frame. Trajectories are shown with the green-blue gradient indicating progression of time (total trajectory length is 128/30 $\approx$ 4s). The purple line is a reconstruction of the trajectory showing the latent space is able to represent this trajectory. The red dots indicate the initial states the model is conditioned on along with the gates pictured. Data includes counter-clockwise, clock-wise, and straight movements from an initial heading allowing for stronger generalization.

D Training Details

All models were trained on a NVIDIA RTX 2080 Ti for 400 epochs. The autoencoder model is trained on 221728 samples and takes $\approx$ 16 hours to fully converge although after 1.5 hours of training the model sees good performance. The diffusion models are trained on 151544 samples and take $\approx$ 12 hours to fully converge. We also train the a second Diffusion Model with an exponential moving average function of the model updated with the optimizer.

D.1 Autoencoder

Table 5: Training hyperparameters used for autoencoder models.

Hyperparameter	Value
Learning Rate	0.0002
Batch Size	128
Batches Per Step	4
KL Divergence β	0.05
K	2
(ζ_1, ζ_2)	(0.3, 0.1)
Horizon Weighting	$w(j) = \begin{cases} 3.0, & \text{if } j \leq 0.1H \\ 1.0, & \text{otherwise} \end{cases}$

D.2 Diffusion Model and Latent Diffusion Model

Table 6: Training hyperparameter used for diffusion models.

Hyperparameter	Value
Learning Rate	0.0001
Batch Size	128
Batches Per Step	4
Loss	L1
EMA Decay	0.995
Batches per EMA	20
Batches before EMA	10
Training Objective	v
Diffusion Timesteps	100
Conditioning Dropout	0.2
ϕ_{c*}	$\begin{pmatrix} 5 & 5 & 5 & 5 \end{pmatrix}$

E Network Architecture

E.1 Variational Autoencoder

Encoder

- **Input Transformation:** The input $[B, H, D]$ is first rearranged to shape $[B, D, H]$ and passed through a `LinearAttention` block followed by a `Conv1dNormalized` layer that maps from D to the base number of feature channels F.
- **Downsampling Blocks:** The encoder consists of a sequence of L downsampling blocks, each composed of:
  1. `ResNet1DBlock` from C_{in} to C_{out}
  2. `LinearAttention` with 4 heads and dimension per head 8
  3. `ResNet1DBlock` ($C_{\text{out}} \to C_{\text{out}}$)

4. `Downsample1d` (reduces temporal length by a factor of 2) and the number of channels doubles

- **Latent Projection:** After the final downsampling stage, a `ResNet1DBlock` is applied. Then, two separate 1×1 convolutions produce the latent distribution parameters μ and $\log \sigma^2$ of shape $[B, L', Z]$, where L' is the compressed horizon and Z is the latent dimension.

Decoder

- **Initial Projection:** The latent input $z \in \mathbb{R}^{B \times L' \times Z}$ is rearranged and passed through a `Conv1dNormalized` layer to lift it to C feature channels.
- **Upsampling Blocks:** The decoder mirrors the encoder with a series of L upsampling blocks, each composed of:
  1. `ResNet1DBlock` ($C_{\text{in}} \to C_{\text{in}}$)
  2. `LinearAttention`
  3. `ResNet1DBlock` ($C_{\text{in}} \to C_{\text{out}}$)
  4. `Upsample1d` (doubles temporal resolution)
- **Final Projection:** After the last upsampling block, a `ResNet1DBlock` and a 1×1 convolution project the features back to the trajectory dimension D. A final `LinearAttention` layer is applied to add small corrections.

E.2 Diffusion Model

Input Preprocessing

- The input tensor $z \in \mathbb{R}^{B \times H \times D}$ is rearranged to shape $[B, D, H]$.
- A sinusoidal positional embedding is computed from scalar time $t \in \mathbb{R}^B$.
- This embedding is passed through a multi-layer perceptron (MLP):

$$t_{\text{embed}} = \text{MLP}(\text{SinusoidalPosEmb}(t)) \in \mathbb{R}^{B \times F}$$

  where F is the feature dimension.

Downsampling Path The U-Net consists of L downsampling stages. Each stage includes:

- Two `ResNet1DBlock` modules with shared time embedding input.
- `Downsample1d` that reduces the temporal resolution by a factor of 2 (except at final stage) and doubles the number of channels.

Bottleneck (Middle Block)

- One `ResNet1DBlock`.
- Optional `LinearAttention`.
- Another `ResNet1DBlock`.

Conditioning Mechanism The model receives two conditioning inputs:

- Prior state context $c^* \in \mathbb{R}^{B \times 6 \times 3}$
- Future goal context $x_p \in \mathbb{R}^{B \times 4 \times 4}$

For each:

- A linear embedding transforms it to dimension $4F$.

- Sinusoidal position embeddings are added to each timestep of the context.
- The context is rearranged to match the attention format and processed using `CrossAttention`.

Upsampling Path

- For each upsampling stage, the input is concatenated with the corresponding skip connection from the downsampling path.
- Each stage includes:
  - Two `ResNet1DBlock` modules.
  - Optional `LinearAttention`.
  - `Upsample1d` that doubles the temporal resolution and halves the number of features.

Final Projection

- A `Conv1dNormalized` followed by a 1×1 convolution projects the features back to the original dimension D.
- Output is rearranged to shape $[B \times H \times D]$.